

1. Delete a node from a binary search tree.

```
class Solution {

    public TreeNode deleteNode(TreeNode root, int key)

    { if (root == null) return null;

      if (key < root.val) {
          root.left = deleteNode(root.left, key);
      }
      else if (key > root.val) {
          root.right = deleteNode(root.right, key);
      }
      else {

          if (root.left == null) return root.right;
          if (root.right == null) return root.left;

          TreeNode successor = findMin(root.right);
          root.val = successor.val;
          root.right = deleteNode(root.right, successor.val);
      }

      return root;
    }

    private TreeNode findMin(TreeNode node)
    { while (node.left != null) {
        node = node.left;
    }
    return node;
    }
}
```

2. Check if a binary tree is a binary search tree.

```

class Solution {

    public boolean isValidBST(TreeNode root)
    { return validate(root, null, null);
    }

    private boolean validate(TreeNode node, Integer min, Integer max)

    { if (node == null) return true;

      if ((min != null && node.val <= min) ||
          (max != null && node.val >= max))
          { return false;
          }

      return validate(node.left, min, node.val) && validate(node.right,
node.val, max);
    }
}

```

3. Find the kth smallest and kth largest elements in a BST.

```

class Solution
{ List<Integer>
arr;

    public void inorder(TreeNode
root){ if(root == null) return;

    inorder(root.left);
    arr.add(root.val);
    inorder(root.right);
}

    public int kthSmallestAndBiggest(TreeNode root, int k)
    { arr= new ArrayList<>();
      inorder(root);
      return new int[]{arr.get(k-1), arr.get(n-k-1)};
    }
}

```

4. Find whether the given BST is balanced or not

```
class Solution
{
    boolean ans =
    true;
    public int height(TreeNode root)
        { if (root == null) return 0;

        int left = height(root.left);
        int right = height(root.right);

        if (Math.abs(left - right) > 1) ans = false;

        return 1 + Math.max(left, right);
    }

    public boolean isBalanced(TreeNode root)
        { height(root);
        return ans;
    }
}
```